University of Stirling

ITNPBD2 Representing and Manipulating Data

Assignment Autumn 2025

A Consultancy Job for JC Penney

This notebook forms the assignment instructions and submission document of the assignment for ITNPBD2. Read the instructions carefully and enter code into the cells as indicated.

You will need these five files, which were in the Zip file you downloaded from the course webpage:

- jcpenney_reviewers.json
- jcpenney_products.json
- products.csv
- reviews.csv
- users.csv

The data in these files describes products that have been sold by the American retail giant, JC Penney, and reviews by customers who bought them. Note that the product data is real, but the customer data is synthetic.

Your job is to process the data, as requested in the instructions in the markdown cells in this notebook.

Completing the Assignment

Rename this file to be xxxxxx_BD2 where xxxxxx is your student number, then type your code and narrative description into the boxes provided. Add as many code and markdown cells as you need. The cells should contain:

- **Text narrative describing what you did with the data**
- **The code that performs the task you have described**
- **Comments that explain your code**

The final structure (in PDF) of your report must:

- **Start from the main insights observed (max 5 pages)**
- **Include as an appendix the source code used for producing those insights (max 15 pages)**
- **Include an AI cover sheet (provided on Canvas), which must contain a link to a versioned notebook file in OneDrive or another platform for version checks.**

Marking Scheme

The assessment will be marked against the university Common Marking Scheme (CMS)

Here is a summary of what you need to achieve to gain a grade in the major grade bands:

Grade	Requirement
Fail	You will fail if your code does not run or does not achieve even the basics of the task. You may also fail if you submit code without either comments or a text explanation of what the code does.
Pass	To pass, you must submit sufficient working code to show that you have mastered the basics of the task, even if not everything works completely. You must include some justifications for your choice of methods, but without mentioning alternatives.
Merit	For a merit, your code must be mostly correct, with only small problems or parts missing, and your comments must be useful rather than simply re-stating the code in English. Most choices for methods and structures should be explained and alternatives mentioned.
Distinction	For a distinction, your code must be working, correct, and well commented and shows an appreciation of style, efficiency and reliability. All choices for methods and structures are concisely justified and alternatives are given well thought considerations. For a distinction, your work should be good enough to present to executives at the company.

The full details of the CMS can be found here

https://www.stir.ac.uk/about/professional-services/student-academic-and-corporate-services/academic-registry/academic-policy-and-practice/quality-handbook/assessment-policy-and-procedure/appendix-2-postgraduate-common-marking-scheme/

Note that this means there are not certain numbers of marks allocated to each stage of the assignment. Your grade will reflect how well your solutions and comments demonstrate that you have achieved the learning outcomes of the task.

Submission

When you are ready to submit, **print** your notebook as PDF (go to File -> Print Preview) in the Jupyter menu. Make sure you have run all the cells and that their output is displayed. Any lines of code or comments that are not visible in the pdf should be broken across several lines. You can then submit the file online.

Late penalties will apply at a rate of three marks per day, up to a maximum of 7 days. After 7 days you will be given a mark of 0. Extensions will be considered under acceptable circumstances outside your control.

Academic Integrity

This is an individual assignment, and so all submitted work must be fully your own work.

The University of Stirling is committed to protecting the quality and standards of its awards. Consequently, the University seeks to promote and nurture academic integrity, support staff academic integrity, and support students to understand and develop good academic skills that facilitate academic integrity.

In addition, the University deals decisively with all forms of Academic Misconduct.

Where a student does not act with academic integrity, their work or behaviour may demonstrate Poor Academic Practice or it may represent Academic Misconduct.

Poor Academic Practice

Poor Academic Practice is defined as: "The submission of any type of assessment with a lack of referencing or inadequate referencing which does not effectively acknowledge the origin of words, ideas, images, tables, diagrams, maps, code, sound and any other sources used in the assessment."

Academic Misconduct

Academic Misconduct is defined as: "any act or attempted act that does not demonstrate academic integrity and that may result in creating an unfair academic advantage for you or another person, or an academic disadvantage for any other member or member of the academic community."

Plagiarism is presenting somebody else's work as your own **and includes the use of artificial intelligence tools beyond AIAS Level 2 or the use of Large Language Models.**. Plagiarism is a form of academic misconduct and is taken very seriously by the University. Students found to have plagiarised work can have marks deducted and, in serious cases, even be expelled from the University. Do not submit any work that is not entirely your own. Do not collaborate with or get help from anybody else with this assignment.

The University of Stirling's full policy on Academic Integrity can be found at:

https://www.stir.ac.uk/about/professional-services/student-academic-and-corporate-services/academic-registry/academic-policy-and-practice/quality-handbook/academic-integrity-policy-and-academic-misconduct-procedure/

The Assignment

Your task with this assignment is to use the data provided to demonstrate your Python data manipulation skills.

There are three `.csv` files and two `.json` files so you can process different types of data. The files also contain unstructured data in the form of natural language in English and links to images that you can access from the JC Penney website (use the field called `product_image_urls`).

Start with easy tasks to show you can read in a file, create some variables and data structures, and manipulate their contents. Then move onto something more interesting.

Look at the data that we provided with this assessment and think of something interesting to do with it using whatever libraries you like. Describe what you decide to do with the data and why it might be interesting or useful to the company to do it.

You can add additional data if you need to - either download it or access it using `requests`. Produce working code to implement your ideas in as many cells as you need below. There is no single right answer, the aim is to simply show you are competent in using python for data analysis. Exactly how you do that is up to you.

For a distinction class grade, this must show originality, creative thinking, and insights beyond what you've been taught directly on the module.

Structure

You may structure the appendix of the project how you wish, but here is a suggested guideline to help you organise your work, based on the CRISP-DM data science methodology:

1. **Business understanding** - What business context is the data coming from? What insights would be valuable in that context, and what data would be required for that purporse?
2. **Data understanding and preparation** - Explore the data and show you understand its structure and relations, with the aid of appropriate visualisation techniques. Assess the data quality, which insights you would be able to answer from it, and what preparation the data would require. Add new data from another source if required to bring new insights to the data you already have.

3. **Data modeling (optional)** - Would modeling be required for the insights you have considered? Use appropriate techniques, if so.
4. **Evaluation and deployment** - How do the insights you obtained help the company, and how can should they be adopted in their business? If modeling techniques have been adopted, are their use scientifically sound and how should they be mantained?

Remember to make sure you are working completely on your own.

Don't work in a group or with a friend

In [ ]:
```
# Put your code and comments in cells below here
# Add as many cells as you need
```

Step 1: Data Loading and Initial Inspection

The first section of this code will include loading the data and verifying the structure for future use.

In this step:

- Two json files and the three csv files were loaded.
- A dictionary was used to manage the csvs effectively.
- The record counts were printed.
- The first ten rows of all data were then printed to gain an understanding of structure and content.

This step ensures all data is loaded and visible and some intial information on the dataset is ascertained.

In [ ]:
```
# Load in JCpenny reviewer and product json files
import pandas as pd

products_json_df = pd.read_json("jcpenney_products.json", lines=True)
reviewers_json_df = pd.read_json("jcpenney_reviewers.json", lines=True)

# Preview the structure and confirm row counts

display(products_json_df.head())
display(reviewers_json_df.head())

print("Products JSON rows:", len(products_json_df))
print("Reviewers JSON rows:", len(reviewers_json_df))

# Load CSV data and display row counts
csv_files = {
```

```
    "Products": pd.read_csv("products.csv"),
    "Reviews": pd.read_csv("reviews.csv"),
    "Users": pd.read_csv("users.csv")
}

for name, df in csv_files.items():
    print(f"{name}: {len(df)} rows")

# Unpack DataFrames for later use
products_csv_df, reviews_csv_df, users_csv_df = csv_files.values()

# Preview the first 10 rows of each CSV to inspect structure
for name, df in csv_files.items():
    print(f"\n{name} Preview:")
    display(df.head(10))
```

Step 2: Data Understanding and Quality Check

Before cleaning and merging, the datasets are explored to assess the structure and quality.

In this step:

- Column names are reviewed for each dataset.
- Missing values are identified and summarised.
- A visual heatmap highlights patterns of missing data.

This step ensures that any inconsistencies or incomplete records can be addressed before further analysis.

In [ ]:
```
# Step 2: Data Understanding and Quality Check

# Creating a dictionary to store all data frames
all_datasets = {
    "Products CSV": products_csv_df,
    "Reviews CSV": reviews_csv_df,
    "Users CSV": users_csv_df,
    "Products JSON": products_json_df,
    "Reviewers JSON": reviewers_json_df
}

# Loop through and print the column names for each dataset
for name, df in all_datasets.items():
    print(f"\n{name} Columns ({len(df.columns)} total):")
    print(df.columns.tolist())

# Missing Values Summary
print("\nMissing values in Products CSV:")
print(products_csv_df.isna().sum())

print("\nMissing values in Reviews CSV:")
print(reviews_csv_df.isna().sum())

print("\nMissing values in Users CSV:")
print(users_csv_df.isna().sum())
```

```
print("\nMissing values in jcpenney_products.json:")
print(products_json_df.isna().sum().sort_values(ascending=False))

print("\nMissing values in jcpenney_reviewers.json:")
print(reviewers_json_df.isna().sum().sort_values(ascending=False))

# Visualise missing data for the identified data set with data missing
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 4))
sns.heatmap(products_csv_df.isna(), cbar=False, cmap="coolwarm")
plt.title("Missing Values in Products CSV")
plt.show()
```

Step 3: Data Cleaning and Preperation After assessing the data structure and quality, the next step is to clean and standardise it to ensure consistency across files before merging.

In this step:

- All collumn names were standardised by converting to lowercase and key fields renamed.
- All missing values were flagged
- DOB was changed into age groups for more effective analysis
- Removed duplicates
- Prepare the data for merging by ensuring join keys were consistant

In [ ]:
```
# STEP 3 — Clean and Prepare Data for Analysis
import pandas as pd
import numpy as np
import re

# Standardise column names
def standardise_columns(df):
    df.columns = df.columns.str.strip().str.lower()
    return df

datasets = [products_csv_df, reviews_csv_df, users_csv_df, products_json_df, review
datasets = [standardise_columns(df) for df in datasets]
products_csv_df, reviews_csv_df, users_csv_df, products_json_df, reviewers_json_df

# Rename key identifier columns for consistency
rename_map = {'uniq_id': 'product_id', 'username': 'user_id'}
for df in [products_csv_df, reviews_csv_df, users_csv_df, products_json_df, reviewe
    df.rename(columns={k: v for k, v in rename_map.items() if k in df.columns}, inp

# Clean product pricing information and extract numeric values from list_price and
for col in ['list_price', 'sale_price']:
    if col in products_json_df.columns:
        products_json_df[col] = (
            products_json_df[col]
            .astype(str)
```

```
            .str.extract(r'(\d+\.?\d*)')[0]  # Extract numeric portions like 41.09
            .astype(float)
        )

# Flag missing price values if either list or sale price is missing
products_json_df['price_missing_flag'] = (
    products_json_df[['list_price', 'sale_price']].isna().any(axis=1).astype(int)
)

# Create a discount flag and percentage (only when both prices exist)
if {'list_price', 'sale_price'}.issubset(products_json_df.columns):
    products_json_df['discount_flag'] = (
        (products_json_df['sale_price'] < products_json_df['list_price'])
        & products_json_df['sale_price'].notna()
        & products_json_df['list_price'].notna()
    ).astype(int)

# Discount percentage (only for valid discount cases)
    products_json_df['discount_percent'] = np.where(
        products_json_df['discount_flag'] == 1,
        ((products_json_df['list_price'] - products_json_df['sale_price']) /
         products_json_df['list_price'] * 100).round(2),
        np.nan
    )

# Clean user data
if 'dob' in users_csv_df.columns:
    # Handle UK date format (dd.mm.yyyy)
    users_csv_df['dob'] = pd.to_datetime(users_csv_df['dob'], format='%d.%m.%Y', er

    # Calculate age in years
    users_csv_df['age'] = ((pd.Timestamp('today') - users_csv_df['dob']).dt.days /

    # Create age groups
    bins = [0, 25, 40, 60, 100]
    labels = ['<25', '25-40', '40-60', '60+']
    users_csv_df['age_group'] = pd.cut(users_csv_df['age'], bins=bins, labels=label

# Validate cleaning results

print(f"Products missing any price: {products_json_df['price_missing_flag'].sum()}"
print(f"Products with discounts: {products_json_df['discount_flag'].sum()}")
print(f"Users with missing DOBs: {users_csv_df['dob'].isna().sum()}")

# --- 6. Preview cleaned data ---
display(products_json_df[['product_id', 'list_price', 'sale_price',
                          'discount_flag', 'discount_percent', 'price_missing_flag'

display(users_csv_df[['user_id', 'dob', 'age', 'age_group']].head(10))
```

Step 4: Data Analysis and Visualisation

In [ ]:
```
# STEP 4 — Clean and Enhance Reviews Data
import seaborn as sns
import matplotlib.pyplot as plt
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk
import numpy as np
import pandas as pd

# Make sure the sentiment model is available
nltk.download("vader_lexicon", quiet=True)
sia = SentimentIntensityAnalyzer()

#  Inspect and standardise
print(f"Reviews shape: {reviews_csv_df.shape}")
print(f"Columns: {reviews_csv_df.columns.tolist()}")

reviews_csv_df.columns = reviews_csv_df.columns.str.strip().str.lower()

#  Rename columns for clarity
rename_map = {"review": "review_text", "score": "rating"}
reviews_csv_df.rename(columns={k: v for k, v in rename_map.items() if k in reviews_

#  Clean review text and rating values
reviews_csv_df = reviews_csv_df.dropna(subset=["review_text"])
reviews_csv_df = reviews_csv_df[reviews_csv_df["review_text"].str.strip() != ""]

reviews_csv_df["rating"] = pd.to_numeric(reviews_csv_df["rating"], errors="coerce")
reviews_csv_df = reviews_csv_df[reviews_csv_df["rating"].between(0, 5)]

#  Add sentiment score
print("Calculating sentiment scores...")
reviews_csv_df["sentiment_score"] = reviews_csv_df["review_text"].astype(str).apply

#  Summary

print(f"Total valid reviews: {len(reviews_csv_df):,}")
print(f"Average rating: {reviews_csv_df['rating'].mean():.2f}")
print(f"Average sentiment: {reviews_csv_df['sentiment_score'].mean():.2f}")

#  Preview result
display(reviews_csv_df[["user_id", "product_id", "rating", "sentiment_score", "revi
```

In [ ]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Merge reviews with products
merged_reviews_products = reviews_csv_df.merge(
    products_json_df,
    on="product_id",
    how="left"
)
print(f"Merged reviews + products: {len(merged_reviews_products):,} rows")
```

```python
# Merge with users
merged_all = merged_reviews_products.merge(
    users_csv_df,
    on="user_id",
    how="left"
)
print(f"Final merged dataset: {len(merged_all):,} rows")

# Create base persona dataset (aggregate user-level data)
persona_df = (
    merged_all.groupby("user_id")
    .agg({
        "rating": "mean",
        "sentiment_score": "mean",
        "age": "mean",
        "product_id": "count",
        "sale_price": "mean",
        "category": lambda x: x.mode().iloc[0] if not x.mode().empty else np.nan
    })
    .reset_index()
)

# Rename columns for clarity
persona_df.rename(columns={
    "rating": "avg_rating",
    "sentiment_score": "avg_sentiment",
    "age": "avg_age",
    "product_id": "review_count",
    "sale_price": "avg_price",
    "category": "fav_category"
}, inplace=True)


display(persona_df.head(10))

# Check for missing values
print("\nMissing values summary:")
print(persona_df.isna().sum())

# Visualise sentiment vs rating with a regression line
sns.scatterplot(
    data=persona_df,
    x="avg_sentiment",
    y="avg_rating",
    color="steelblue",
    alpha=0.4,
    edgecolor=None
)

sns.regplot(
    data=persona_df,
    x="avg_sentiment",
    y="avg_rating",
    scatter=False,
    color="darkred",
    line_kws={"linewidth": 2}
```

```
)

plt.title("Sentiment vs. Rating", fontsize=14, weight="bold")
plt.xlabel("Average Sentiment (-1 to +1)")
plt.ylabel("Average Rating (0–5)")

plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

In [ ]:

```
import os

#  🦴 Add your MiKTeX path manually here:
miktex_path = r"C:\Users\jess\AppData\Local\Programs\MiKTeX\miktex\bin\x64"

# Add it to the system PATH for this Jupyter session
os.environ["PATH"] += os.pathsep + miktex_path

# Test whether Jupyter can now see it
!xelatex --version

import os

# Tell Jupyter where MiKTeX's xelatex.exe lives
os.environ["PATH"] += os.pathsep + r"C:\Users\jess\AppData\Local\Programs\MiKTeX\mi

# Confirm it’s visible
!where xelatex
```

In [ ]:

```
!pip install nbconvert[webpdf]
!jupyter nbconvert --to webpdf "BD2_Assignment - 2718233.ipynb" --allow-chromium-do
```